

2 — Infrastructure, Data, and Maintenance

What this document is: the operational detail behind each piece of infrastructure — database schema, hosting configuration, AI models, the CMS — plus how to maintain, upgrade, and troubleshoot each one.

Verified against the codebase on 2026-09-07.

2.1 Database (Supabase Postgres)

Connection and identity

Item	Value
Project ref	clryhwqaghvdikgesjbc
URL	https://clryhwqaghvdikgesjbc.supabase.co
Extensions	pgvector (embeddings), plus Supabase defaults
Migrations	supabase/migrations/ — 36 files, numbered 001–033 plus dated ones

Two keys, used very differently:

- **NEXT_PUBLIC_SUPABASE_ANON_KEY** — browser-safe, RLS-enforced.
- **SUPABASE_SERVICE_ROLE_KEY** — **bypasses RLS entirely**. Server-side and scripts only. Never expose to the client.

Schema by domain (~57 tables)

Identity & access profiles · user_roles · role_change_log · api_keys · beta_access_codes · professional_profiles

Billing subscriptions · stripe_customers · stripe_events · referral_codes · referral_events

Academy (see §2.2 for the dual-source model) courses · tracks · lessons · quizzes · quiz_questions · quiz_options · audio_slots · course_enrollments · course_player_enrollments · course_lesson_progress · course_quiz_attempts · module_progress · learning_tracks · user_learning_paths · lesson_notes · lesson_bookmarks · learner_audio_uploads · certifications

AI / RAG rag_documents (vectors) · rag_query_log · rag_feedback · conversations · conversation_messages

Health data hospitals · hospitals_cms · state_health_metrics · state_initiatives · state_time_series · hti_scores · national_benchmark · rht_state_profiles

Community & engagement `community_categories` · `community_threads` · `community_posts` · `community_upvotes` · `wire_comments` · `wire_comment_upvotes` · `bookmarks` · `chapter_notes` · `survey_editions` · `survey_responses` · `tester_feedback`

Ops `webhook_inbox` · `ticker_cache` · `advisory_clients` · `advisory_reports`

Key database functions

- **`hybrid_search_rag`** — the RPC the AI calls for every question. Combines vector similarity with keyword search. Defined in migration `007`.

Row-Level Security

RLS policies live in migrations `006_rls_policies.sql` and `008_rls_audit.sql`. The **service-role key bypasses all of them** — which is why scripts using it must be written carefully.

Maintaining the database

Adding a migration

1. Create `supabase/migrations/0NN_description.sql` (next number in sequence).
2. Apply it via the Supabase SQL editor or CLI.
3. Migrations are **not** auto-applied by CI — this is a manual step.

pgvector maintenance Migration `021_pgvector_hnsw_maintenance.sql` covers HNSW index upkeep. If AI retrieval gets slow, reindexing is the first thing to check.

Log pruning `020_rag_query_log_pruning.sql` prunes query logs. Confirm it is actually running if that table grows unexpectedly.

Backups — Supabase's own backup tooling. Verify the retention setting on your plan; free-tier retention is short.

2.2 Content management (Sanity)

Item	Value
Project ID	<code>fxz10x17</code>
Dataset	<code>production</code>
API version	pinned via <code>NEXT_PUBLIC_SANITY_API_VERSION</code>
Studio	Embedded at <code>/studio</code> in the Next.js app
Schemas	<code>frontend/sanity/schemaTypes/</code> — 24 types
Document count	~250 <code>academyModule</code> docs alone

Tokens

- **NEXT_PUBLIC_SANITY_PROJECT_ID** / **_DATASET** / **_API_VERSION** — public, safe.
- **SANITY_API_TOKEN** — write token. Server-side only.

The Academy dual-source model (critical)

Academy lessons are assembled from **two databases**:

Supabase lessons row — sanity_slug —▶ Sanity academyModule doc

(structure, order, (the actual rich body)

progress, quizzes)

Rule: if `lessons.sanity_slug` is null, the app shows thin legacy `content_blocks` from the Supabase row instead of the Sanity content. Writing content in Sanity is only half the job — the slug must be linked.

Task	Command (run from frontend/)
Audit which lessons are rich vs. thin	<code>node scripts/audit-courses.mjs</code>
Link lessons that have Sanity content	<code>node scripts/link-sanity-slugs.mjs</code> (dry run) → add <code>--commit</code>
Seed/refresh all course structure	<code>node scripts/seed-all-courses.mjs</code>

Seed scripts are **upsert-only** — **they never delete**. Removing a course or lesson means deleting the Supabase rows directly.

Changing a content schema

1. Edit the file in `frontend/sanity/schemaTypes/`.
2. Register it in `schemaTypes/index.ts` if new.
3. Redeploy the frontend (Studio is part of the app).
4. **If the type should be searchable by the AI**, also add a GROQ query for it in `backend/services/indexing.py` → `SANITY_QUERIES`, then re-run ingestion. Otherwise the AI will never see it.

2.3 Hosting

Frontend — Vercel

Setting	Value
Framework	Next.js 16.2.7 (Turbopack)
Region	iad1
Deploy trigger	Push to main (automatic)

Setting	Value
Config	<code>frontend/vercel.json</code>

Environment variables are set in the Vercel dashboard, **not** in the repo.

Backend — Fly.io

Setting	Value
App	<code>vhp-backend</code>
Region	<code>sjc</code>
Machine	shared-1x-CPU, 1 GB RAM, 1 machine
URL	<code>https://vhp-backend.fly.dev</code>
Config	<code>backend/fly.toml</code>
Deploy	GitHub Action, only on <code>backend/**</code> changes

Scale-to-zero is currently enabled:

```
auto_stop_machines = 'stop'
```

```
auto_start_machines = true
```

```
min_machines_running = 0
```

Cost $\approx$ **\$1–3/month**, at the price of a **5–15 second cold start** on the first chat request after an idle period.

To go back to always-on (no cold start, $\approx$ \$5.70/month), set `min_machines_running = 1` and `auto_stop_machines = 'off'`, then redeploy.

Common Fly commands:

```
flyctl status -a vhp-backend
```

```
flyctl logs -a vhp-backend
```

```
flyctl deploy -a vhp-backend # manual deploy
```

```
flyctl secrets list -a vhp-backend
```

```
flyctl secrets set KEY=value -a vhp-backend # triggers redeploy
```

```
curl https://vhp-backend.fly.dev/health
```

⚠ The Docker build context is large (~819 MB, mostly `venv/` and `data/`). Adding those to `.dockerignore` would speed up deploys noticeably.

2.4 AI models and how to change them

All model selection is centralised in `backend/config.py` and `backend/services/llm.py`.

```
MODEL_FREE = "llama-3.1-8b-instant" # Groq
```

```
MODEL_SUBSCRIBER = os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile")
```

```
MODEL_ADVISORY = "claude-sonnet-4-6" # Anthropic
```

```
EMBEDDING_MODEL = "text-embedding-3-small" # OpenAI
```

Fallback chains (`get_llm_for_role`)

Tier	Primary	Falls back to
free / student	Groq 8b	OpenAI <code>gpt-4o-mini</code>
subscriber	Groq 70b	Groq 8b → <code>gpt-4o-mini</code>
advisory / admin	Claude Sonnet	Groq 70b → Groq 8b → <code>gpt-4o-mini</code>

To change a chat model

- **Subscriber tier:** set the `GROQ_MODEL` secret on Fly — no code change.
- **Any other tier:** edit `backend/config.py`, commit, and let the Fly action redeploy.

⚠ To change the embedding model — read this first

Changing `EMBEDDING_MODEL` invalidates the entire existing vector index. Old embeddings are not comparable to new ones. You must:

1. Change the constant.
2. Confirm the vector dimension matches the `rag_documents` column.
3. **Re-run a full ingest** (`POST /api/ingest`) to rebuild every embedding.

Do not change it casually — this is a rebuild, not a swap.

Rebuilding the AI's knowledge index

```
curl -X POST https://vhp-backend.fly.dev/api/ingest \
```

```
-H "Authorization: Bearer $INGEST_SECRET"
```

```
curl https://vhp-backend.fly.dev/api/ingest/status # check progress
```

Also fires automatically via the Sanity webhook when content is published.

Run this after: a backend deploy, a bulk content import, or any embedding model change.

2.5 Environment variables

Frontend (Vercel dashboard + local `.env.local`)

Variable	Purpose	Secret?
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Database URL	No
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Browser DB key (RLS-enforced)	No
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Bypasses RLS	Yes
<code>NEXT_PUBLIC_SANITY_PROJECT_ID / _DATASET / _API_VERSION</code>	CMS connection	No
<code>SANITY_API_TOKEN</code>	CMS write access	Yes
<code>RESEND_API_KEY</code>	Transactional email	Yes
<code>STRIPE_PRICE_*</code> (8 vars)	Plan price IDs	No
<code>PYTHON_BACKEND_URL</code>	Points to the Fly backend	No
<code>ALLOW_AUTH_BYPASS</code>	Skips auth in CI/testing — never true in prod	—

Backend (Fly secrets + local `backend/.env`)

Variable	Purpose
<code>GROQ_API_KEY</code>	Primary chat LLM
<code>ANTHROPIC_API_KEY</code>	Advisory-tier chat
<code>OPENAI_API_KEY</code>	Embeddings, TTS, fallback
<code>GROQ_MODEL</code>	Override subscriber model
<code>SUPABASE_URL / SUPABASE_SERVICE_ROLE_KEY / SUPABASE_JWT_SECRET / SUPABASE_DB_URL</code>	Database + auth verification

Variable	Purpose
<code>SANITY_PROJECT_ID</code> / <code>SANITY_DATASET</code> / <code>SANITY_API_TOKEN</code> / <code>SANITY_API_VERSION</code>	Content ingestion
<code>INGEST_SECRET</code>	Protects <code>/api/ingest</code> and verifies the Sanity webhook
<code>FRONTEND_URL</code>	CORS origin

Rotating a key: update it in *both* places it is used (Vercel dashboard and/or `flyctl secrets set`). Setting a Fly secret triggers a redeploy.

2.6 Upgrading dependencies

Frontend

```
cd frontend
```

```
npm outdated
```

```
npm update          # minor/patch
```

```
npm run smoke       # typecheck + lint + build + bundle budget
```

Watch out for:

- **Next.js majors** — currently on 16 with Turbopack; major upgrades have historically required App Router changes.
- **React 19** — pinned via `overrides` in `package.json`. Several deps must agree on the React version; the overrides block exists for that reason.
- **Tailwind v4 beta** — `@tailwindcss/postcss` and `tailwindcss` are on a `4.0.0-beta` release. Expect churn; upgrade both together.
- **sanity, next-sanity, @sanity/*** are pinned to `latest`, so they move on their own at install time. Consider pinning exact versions if you want reproducible builds.

Backend

```
cd backend
```

```
pip list --outdated
```

```
# edit requirements.txt, then:
```

```
pip install -r requirements.txt
```

```
ruff check .
```

Watch out for: llama-index — the RAG stack is built on it and its APIs change frequently between versions. Test retrieval after any bump.

Always verify after upgrading

```
cd frontend && npm run smoke      # full local gate
```

```
npx playwright test              # E2E smoke
```

2.7 Costs

Service	What you pay for	Approximate
Fly.io	Backend machine-seconds	\$1–3/mo (scale-to-zero)
Vercel	Frontend hosting	\$0 (Hobby) or \$20/mo (Pro)
Supabase	Postgres + pgvector	\$0 (Free) or \$25/mo (Pro)
Sanity	CMS	\$0 on the free tier
OpenAI	Embeddings (cheap) + TTS (pricier per use)	Usage-based
Groq	Chat inference	\$0 (free tier)
Anthropic	Advisory-tier chat	Usage-based
Stripe	Per-transaction	% of revenue
Sentry / Resend / GitHub	Monitoring, email, CI	Free tiers

Verify every figure against the actual billing dashboards. These reflect what the configuration implies, not invoices.

Cost watch-outs:

- **OpenAI TTS** (personalized-learning audio) is the most expensive per-use item. If usage grows, monitor it specifically.
- If the OpenAI balance hits zero you'll see **credit_balance_exhausted** in Fly logs, and **semantic search silently degrades while chat keeps working** — an easy failure to miss.

2.8 Troubleshooting

Symptom	First checks
AI chat returns errors	<code>curl .../health</code> ; <code>flyctl status</code> ; is the machine cold-starting?
AI answers are vague / miss content	Was ingest run? Check <code>credit_balance_exhausted</code> in Fly logs
First chat of the day is slow	Expected — scale-to-zero cold start (5–15 s)
Academy lesson shows thin content	<code>lessons.sanity_slug</code> is null → run <code>link-sanity-slugs.mjs</code>
New Sanity content invisible to AI	Type missing from <code>SANITY_QUERIES</code> , or ingest not run
Login/roles wrong	Check <code>user_roles</code> ; role = highest matching row
Payment didn't grant access	Check <code>stripe_events</code> and the <code>/api/stripe/webhook</code> logs
Build fails in CI but works locally	Usually a missing env var in CI, or a bundle-size budget breach

Health endpoints

`curl https://vhp-backend.fly.dev/health` # backend liveness

`curl https://vhp-backend.fly.dev/api/ingest/status`

2.9 The book pipeline (maintenance)

Golden rule: the `.docx` is generated. Never hand-edit it. Edit `HTR_Book_v42.md` and rebuild.

`./book.sh who` # who currently "holds" the book (edit lock)

`./book.sh mine` # you take it

`./book.sh claude` # hand it to an assistant

`./book.sh check` # show unsynced Google Docs edits, build nothing

`./book.sh build` # rebuild the `.docx`

Raw pipeline (bypasses the safety check — prefer `./book.sh`):

`python3 book-build/check_manuscript.py` # validate first — must be clean

```
python3 book-build/make_reference.py    # rebuild the Word stylesheet
```

```
python3 book-build/build_docx.py HTR_Book_v42.md HTR_Book_v42.docx \  
  
--cover book-build/cover.png
```

Author round-trip: the author edits in Google Docs. Before rebuilding, run `./book.sh check` (wraps `sync_from_gdocs.py`) to surface their edits so they can be folded into the Markdown — otherwise a rebuild overwrites them.

PDF export is a manual, human step: upload the current `.docx` to Google Docs → File → Download → PDF → save over `HTR_Book_v42.pdf`. No local tool reproduces the styling faithfully.

`check_manuscript.py` enforces the structural invariants (chapter/appendix counts, figure numbering, heading consistency, balanced callout fences). Run it before every build; a clean run is the gate.